Factors

factors

R's representation of categorical data. Consists of:

1. A set of **values**
2. An ordered set of **valid levels**

```
eyes <- factor(x = c("blue", "green", "green"),
               levels = c("blue", "brown", "green"))
```




Stored as an integer vector with a levels attribute

```
eyes
## [1] blue  green green
## Levels: blue brown green

unclass(eyes)
## 1 3 3
## attr(,"levels")
## "blue"  "brown" "green"
```



forcats



Simple functions for working with factors.

```
# install.packages("tidyverse")
library(tidyverse)
```




Warm Up

Decide in your group:

Which religions watch the least TV?

Do married people watch more or less TV than single people?



gss_cat

```
library(forcats)
gss_cat
```

A sample of data from the General Social Survey, a long-running US survey conducted by NORC at the University of Chicago.

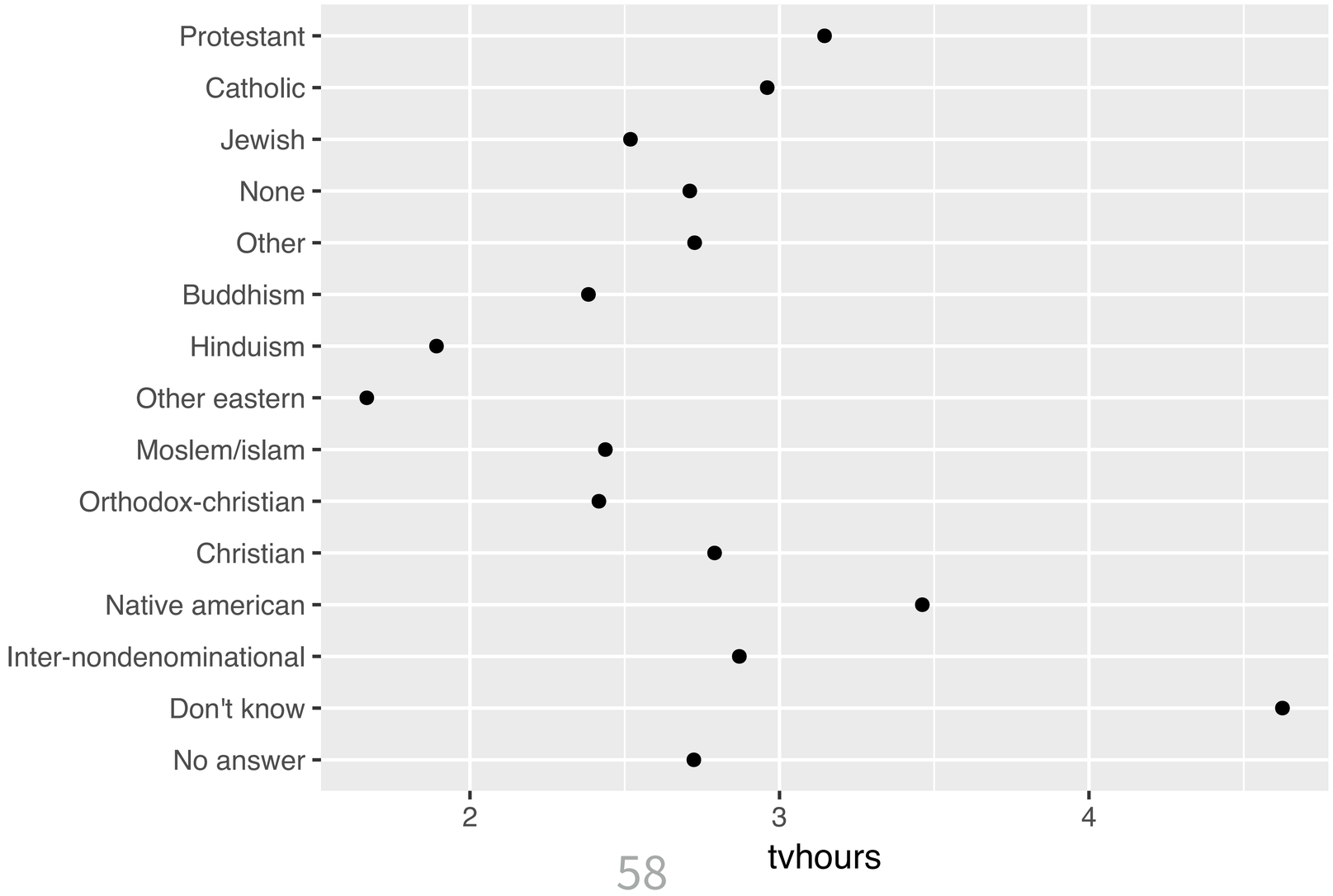
tvhours <int>	marital <fctr>	age <int>	race <fctr>	partyid <fctr>	relig <fctr>
12	Never married	26	White	Ind,near rep	Protestant
NA	Divorced	48	White	Not str republican	Protestant
2	Widowed	67	White	Independent	Protestant
4	Never married	39	White	Ind,near rep	Orthodox-christian
1	Divorced	25	White	Not str democrat	None
NA	Married	25	White	Strong democrat	Protestant
3	Never married	36	White	Not str republican	Christian
NA	Divorced	44	White	Ind,near dem	Protestant
0	Married	44	White	Not str democrat	Protestant

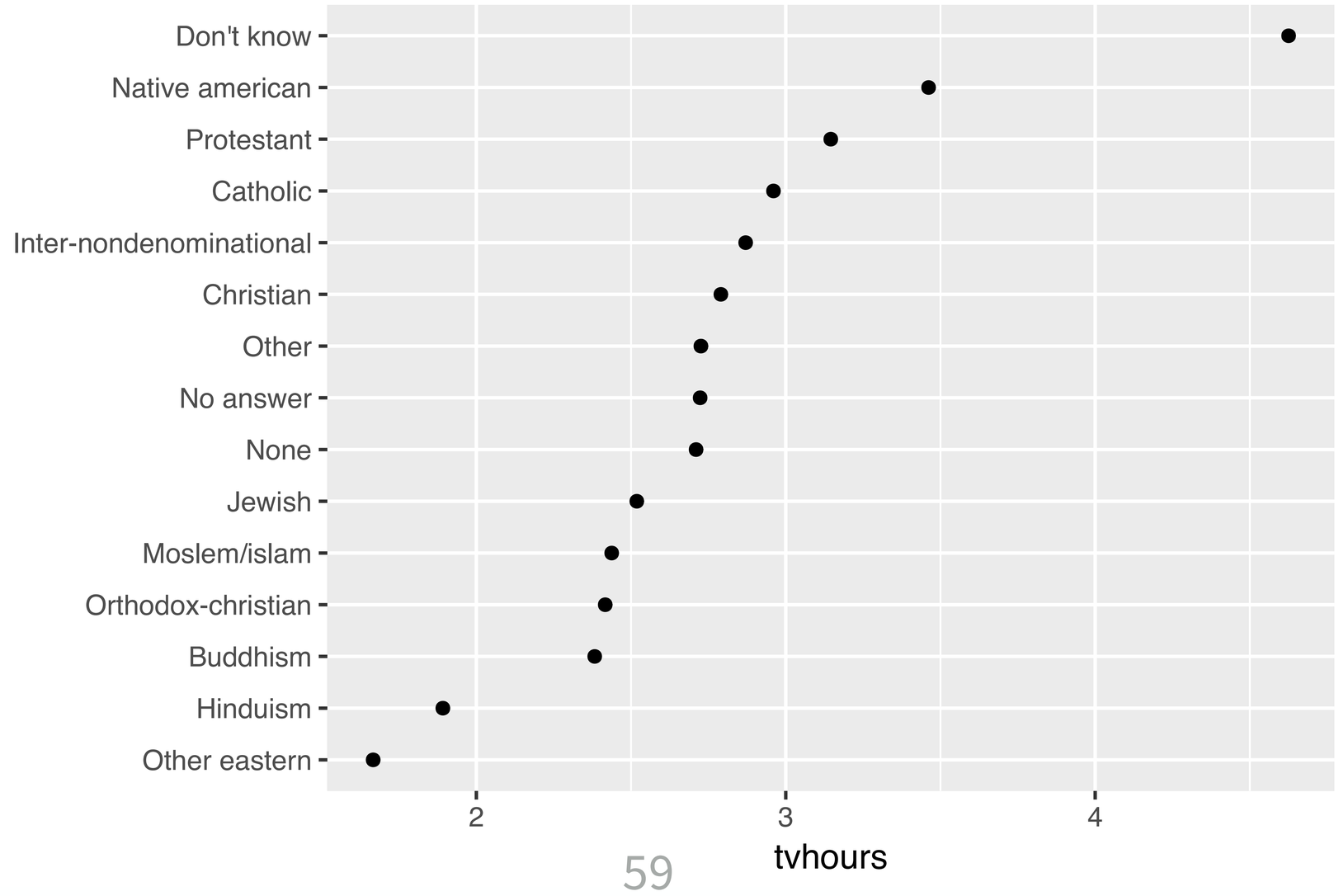


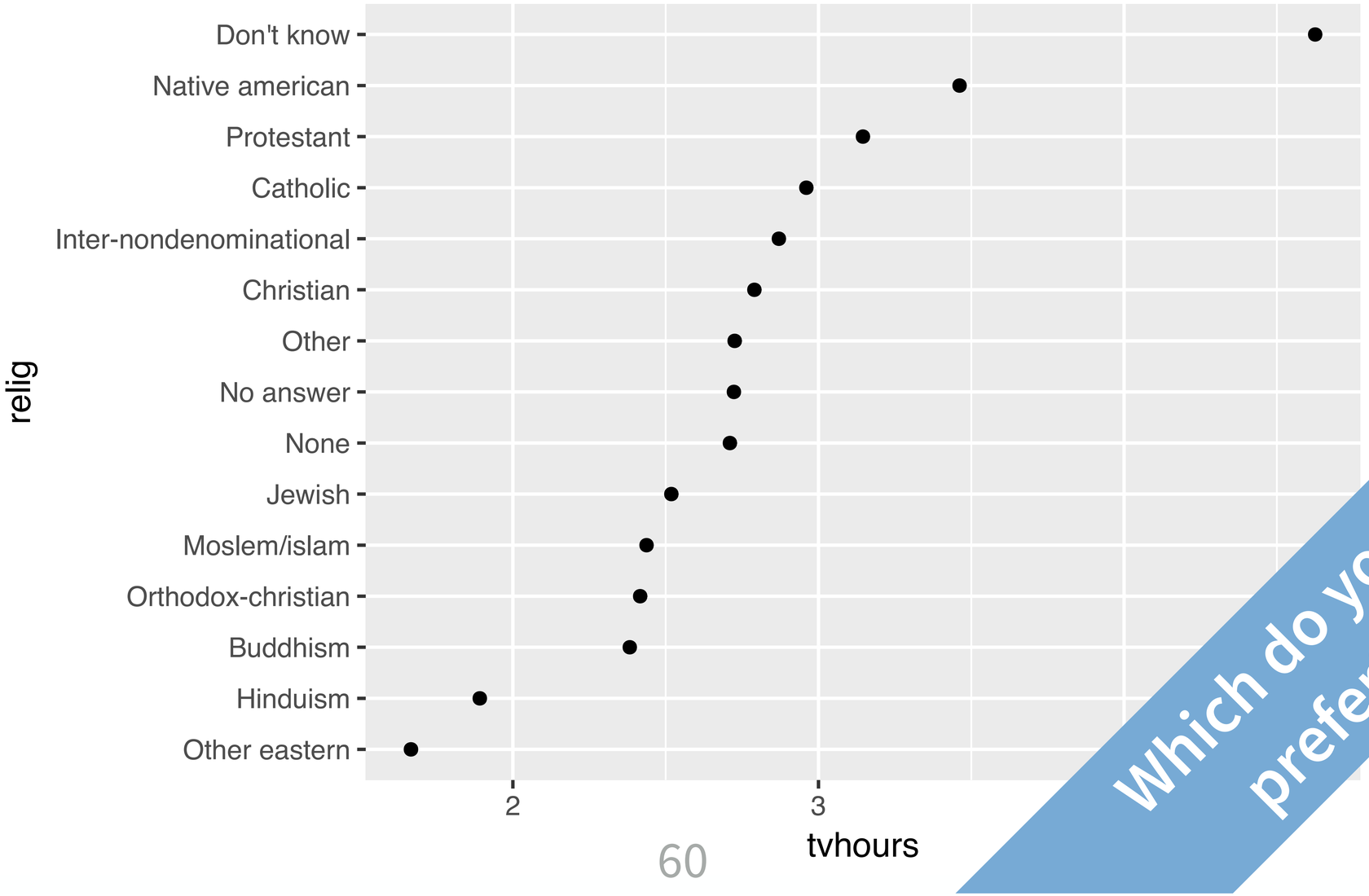
Which religions watch the least TV?

```
gss_cat %>%
  filter(!is.na(tvhours)) %>%
  group_by(relig) %>%
  summarise(tvhours = mean(tvhours)) %>%
  ggplot(aes(tvhours, relig)) +
    geom_point()
```

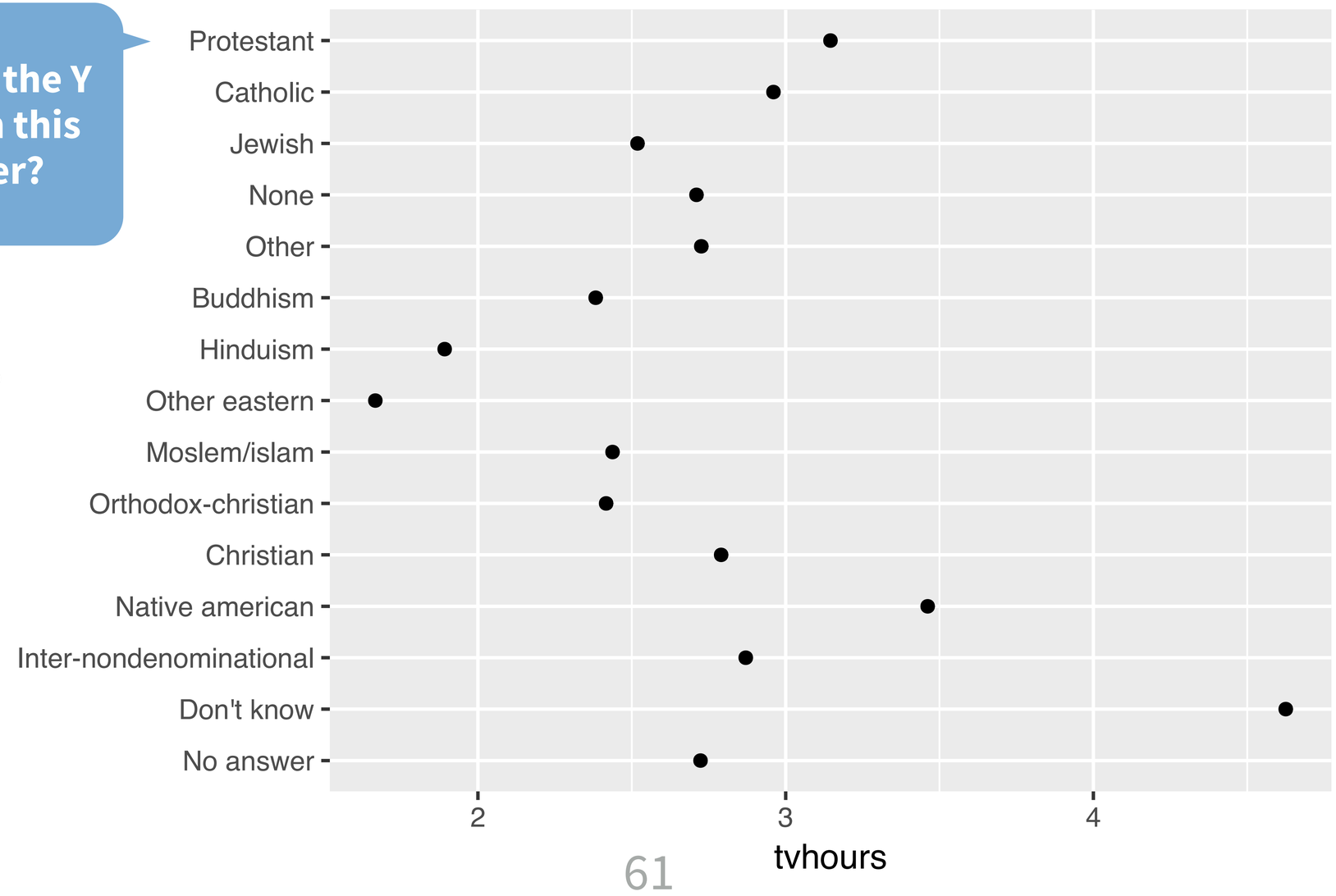



Protestant
Catholic
Jewish
None
Other
Buddhism
Hinduism
Other eastern
Moslem/islam
Orthodox-christian
Christian
Native american
Inter-nondenominational
Don't know
No answer
relig
2
3
4
tvhours


Don't know
Native american
Protestant
Catholic
Inter-nondenominational
Christian
Other
No answer
None
Jewish
Moslem/islam
Orthodox-christian
Buddhism
Hinduism
Other eastern
relig
2
3
4
tvhours

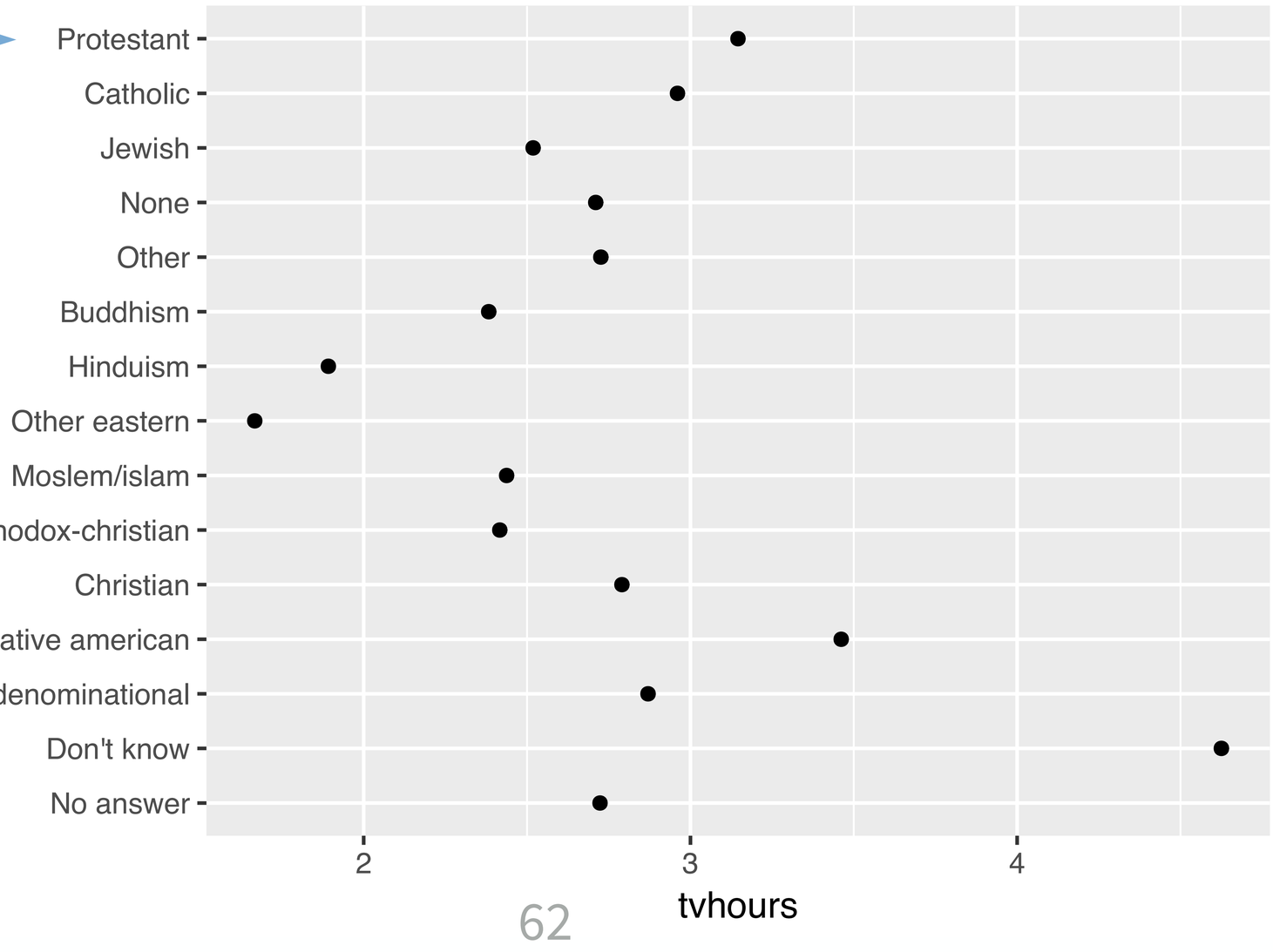

relig
Don't know
Native american
Protestant
Catholic
Inter-nondenominational
Christian
Other
No answer
None
Jewish
Moslem/islam
Orthodox-christian
Buddhism
Hinduism
Other eastern
2
3
tvhours
Which do you prefer?
forcats

Why is the Y axis in this order?



Why is the Y axis in this order?

Because the levels of relig have this order



levels()

Use **levels()** to access a factor's levels

```
levels(gss_cat$relig)
##  [1] "No answer"                "Don't know"
## [3] "Inter-nondenominational" "Native american"
## [5] "Christian"                "Orthodox-christian"
## [7] "Moslem/islam"             "Other eastern"
## [9] "Hinduism"                 "Buddhism"
## [11] "Other"                    "None"
## [13] "Jewish"                   "Catholic"
## [15] "Protestant"               "Not applicable"
```



Most useful skills

1. Reorder the levels
2. Recode the levels
3. Collapse levels



Reordering levels

fct_reorder()

Reorders the levels of a factor based on the result of fun(x) applied to each group of cases (grouped by level).

```
fct_reorder(f, x, fun = median, …, .desc = FALSE)
```

factor to reorder

variable to reorder by (in conjunction with fun)

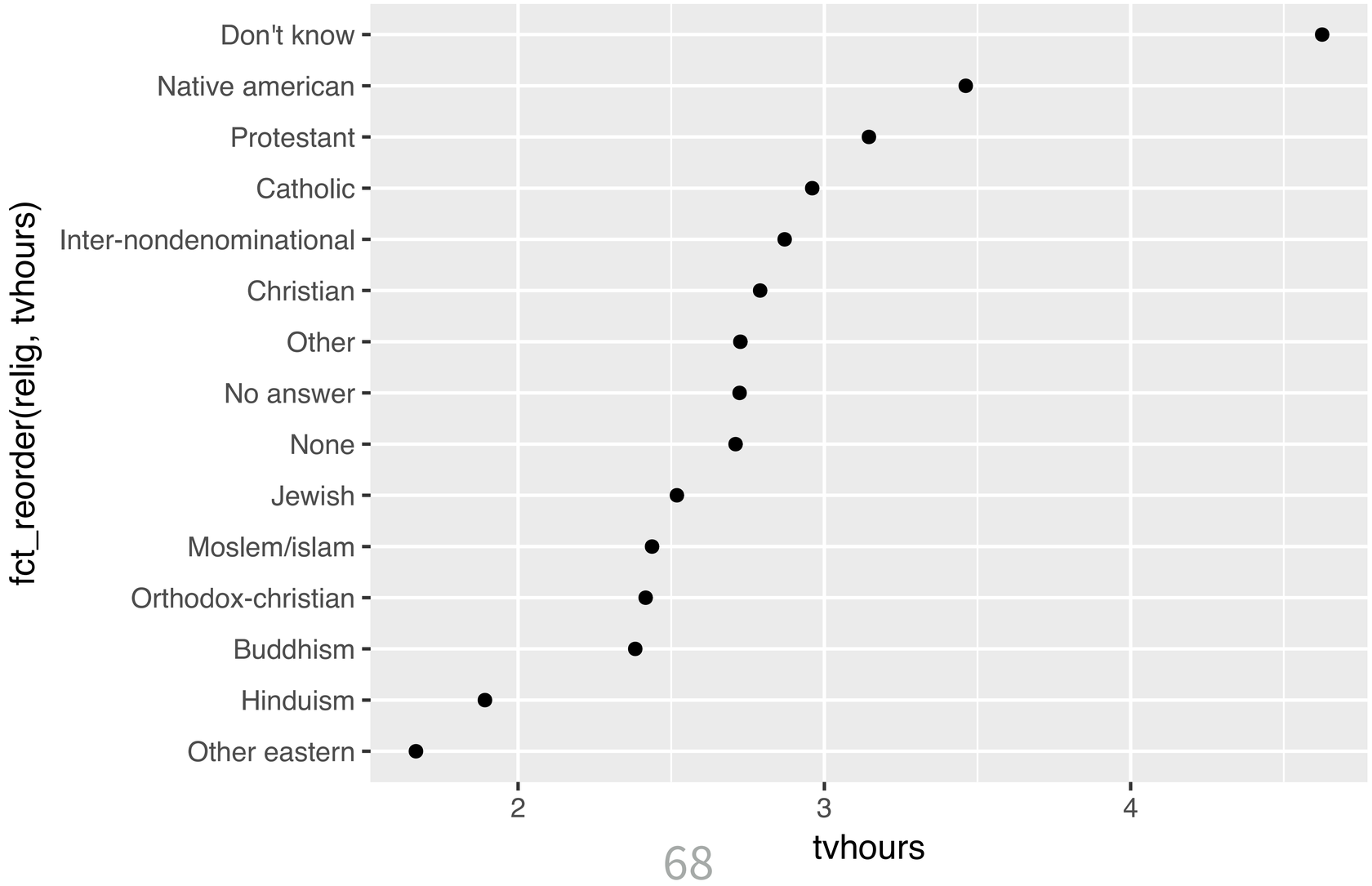
function to reorder by (in conjunction with x)

put in descending order?



```
gss_cat %>%
  filter(!is.na(tvhours)) %>%
  group_by(relig) %>%
  summarise(tvhours = mean(tvhours)) %>%
  ggplot(aes(tvhours, fct_reorder(relig, tvhours))) +
    geom_point()
```

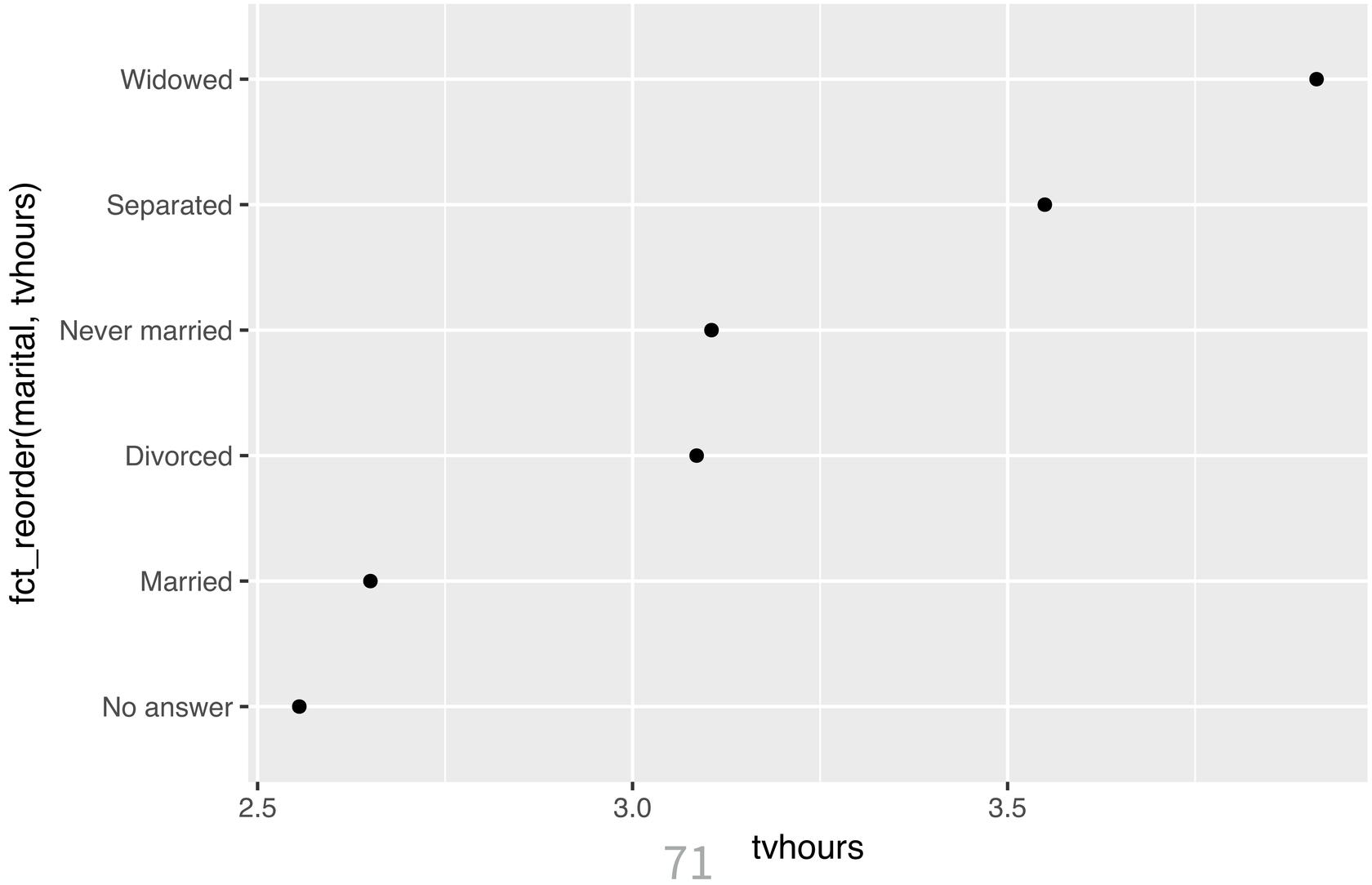




Don't know
Native american
Protestant
Catholic
Inter-nondenominational
Christian
Other
No answer
None
Jewish
Moslem/islam
Orthodox-christian
Buddhism
Hinduism
Other eastern
fct_reorder(relig, tvhours)
2
3
4
tvhours

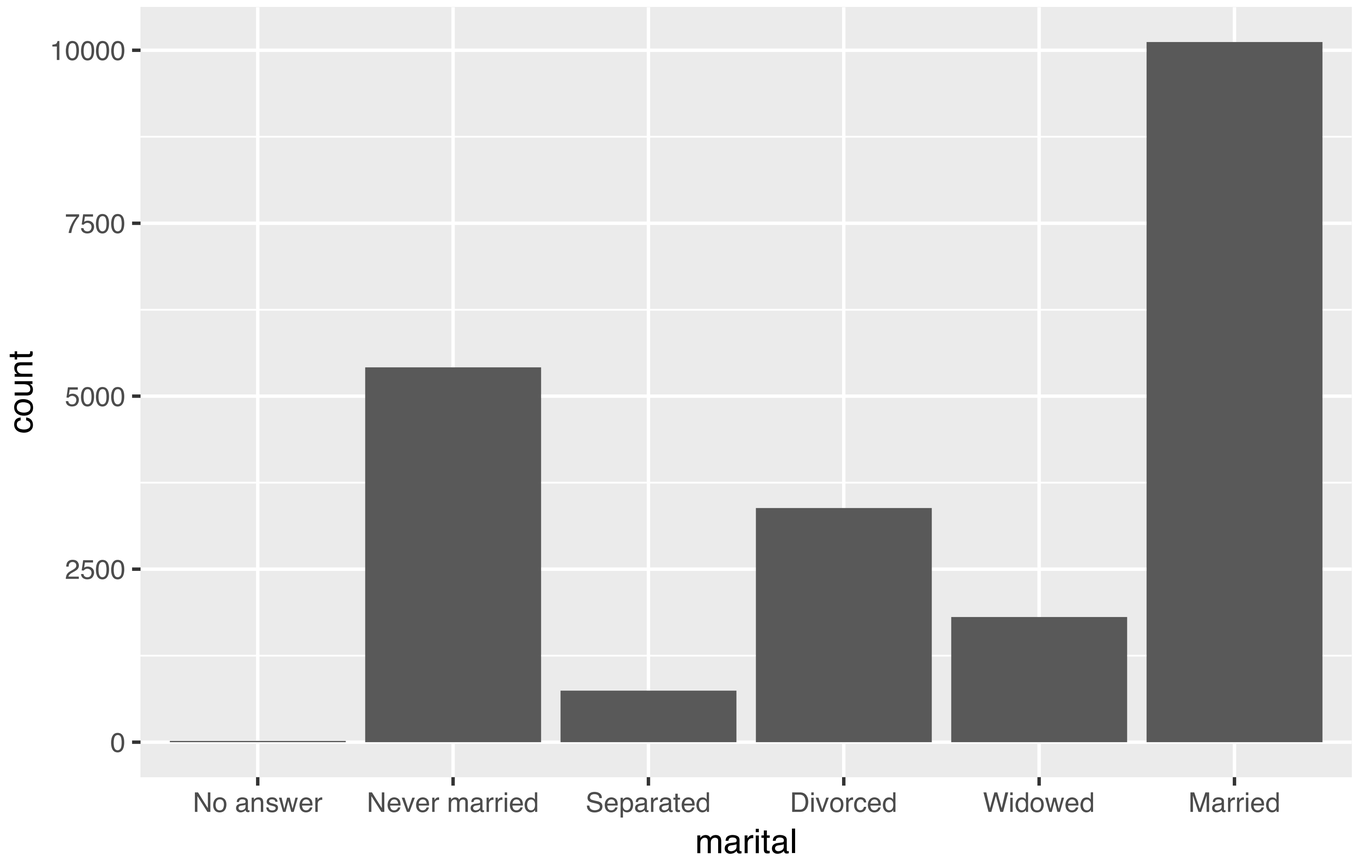
Your Turn 3

Repeat the previous demonstration, some of whose code is in your notebook, to make a sensible graph of average TV consumption by marital status.




Widowed
Separated
Never married
Divorced
Married
No answer
fct_reorder(marital, tvhours)
2.5
3.0
3.5
tvhours

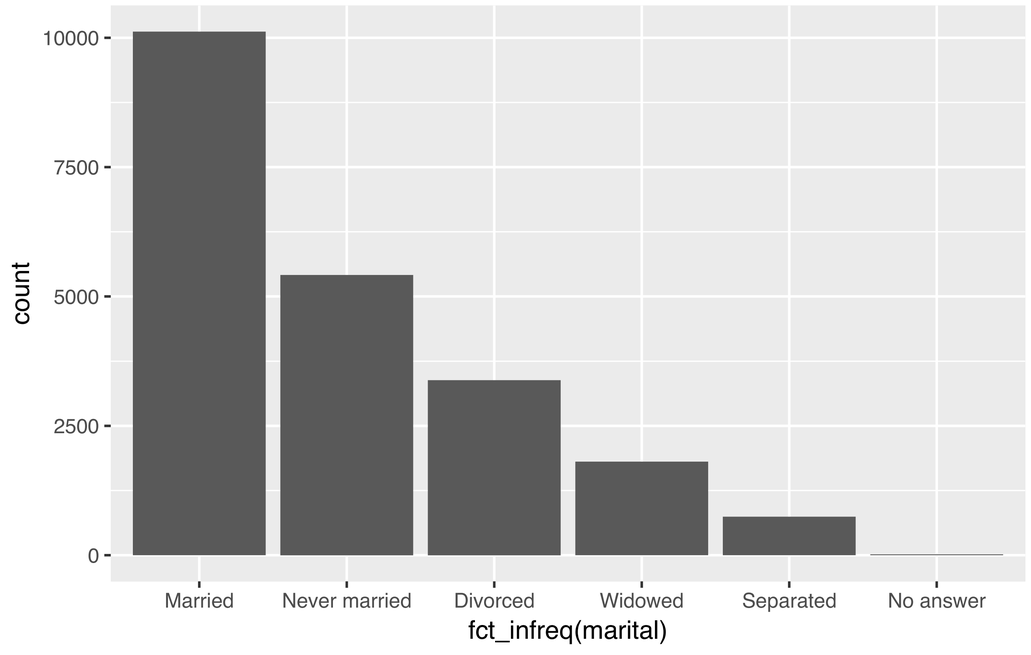
Similar reordering functions




```
gss_cat %>%
  ggplot(aes(marital)) + geom_bar()
```



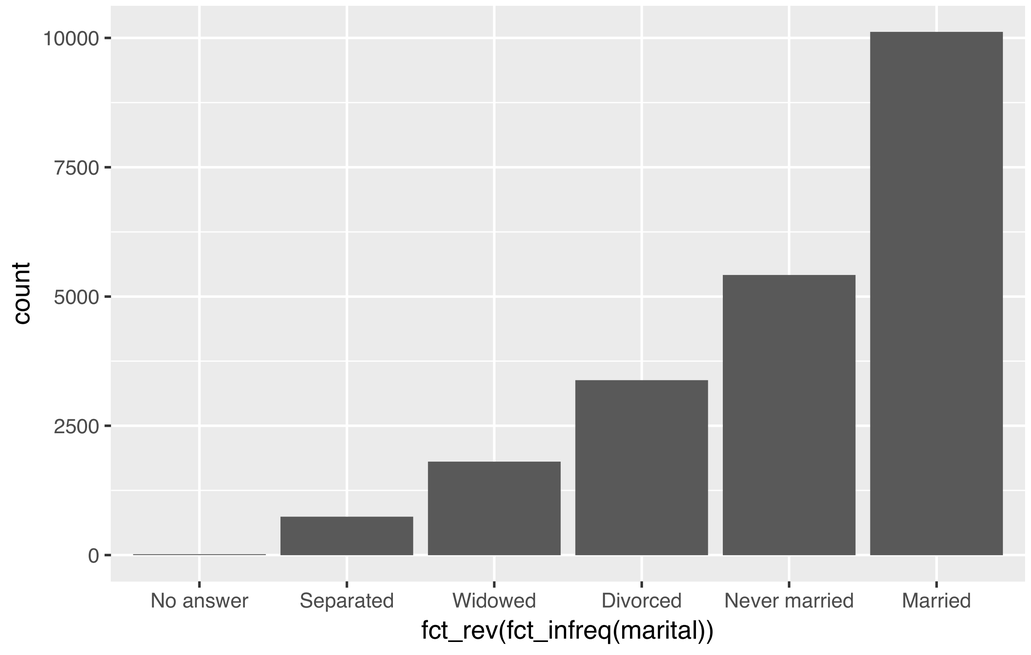
fct_infreq




```
gss_cat %>%
  ggplot(aes(fct_infreq(marital))) + geom_bar()
```



fct_rev



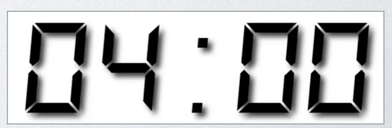

```
gss_cat %>%
  ggplot(aes(fct_rev(fct_infreq(marital)))) + geom_bar()
```

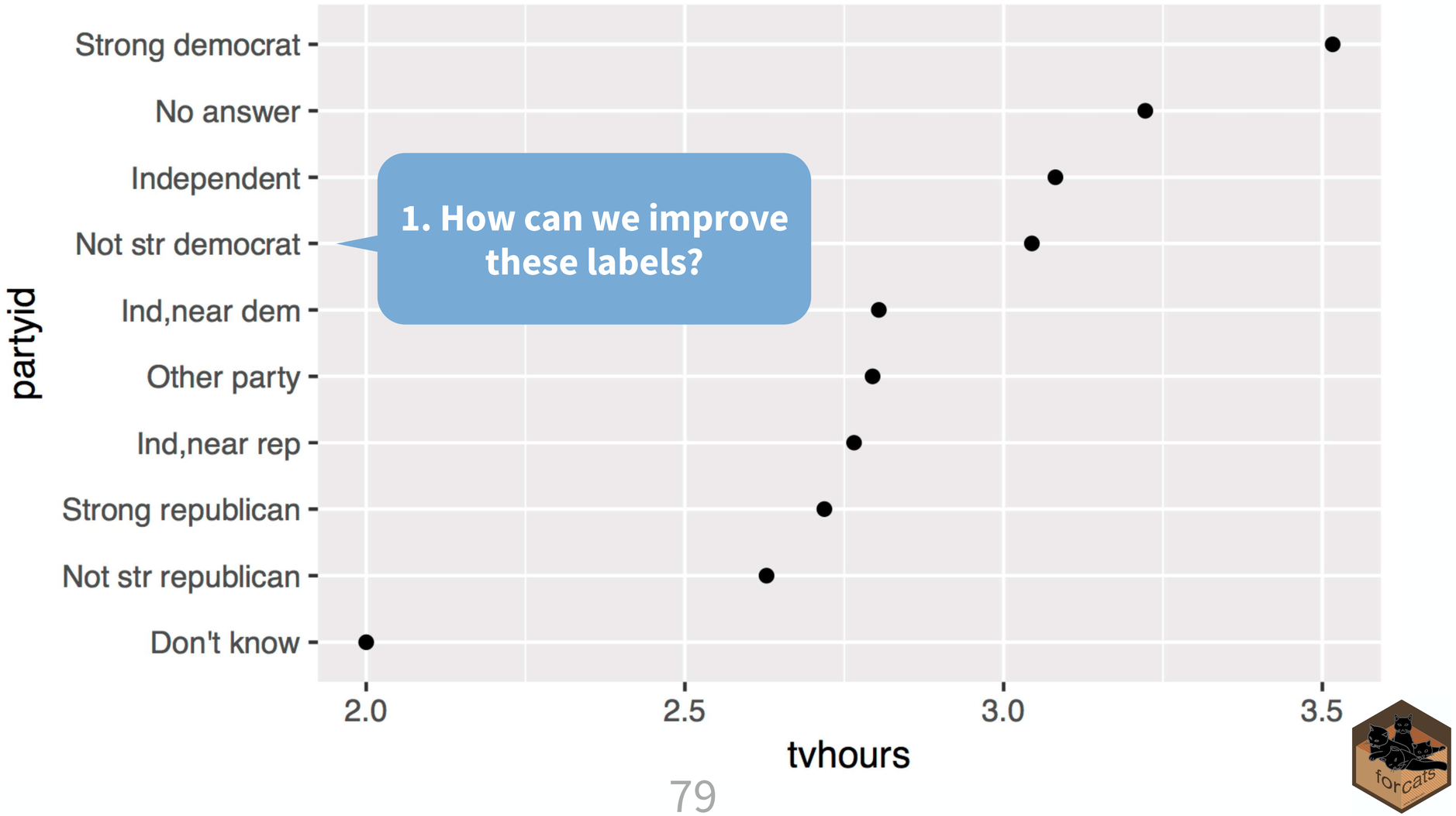
Changing level values

Your Turn 4

Do you think liberals or conservatives watch more TV?

Compute average tv hours by party ID and then plot the results.




Strong democrat
No answer
Independent
Not str democrat
Ind,near dem
Other party
Ind,near rep
Strong republican
Not str republican
Don't know
partyid
2.0
2.5
3.0
3.5
tvhours
1. How can we improve these labels?

fct_recode()

Changes values of levels

```
fct_recode(f, Independent = "Ind,near dem")
```

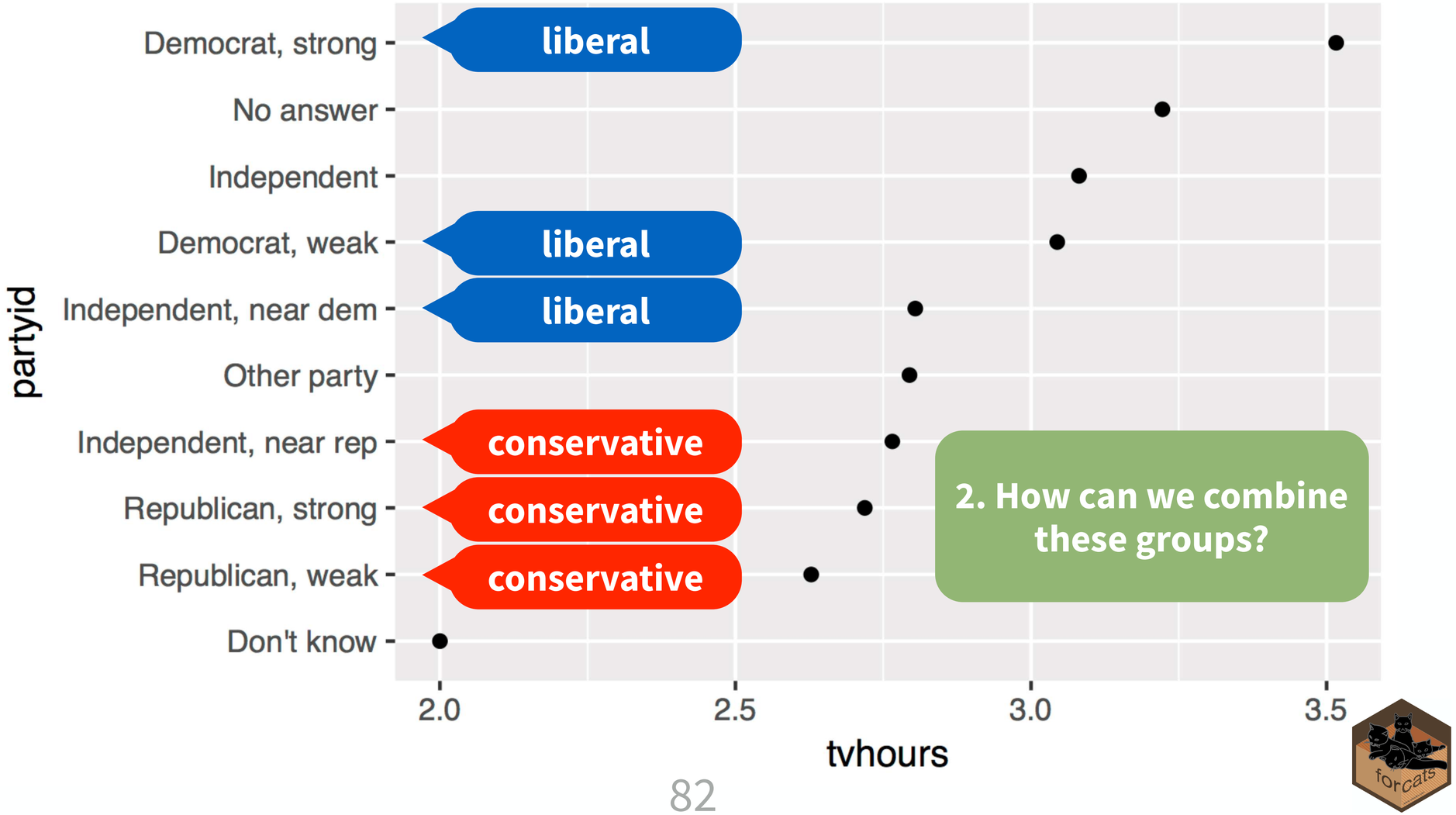
factor with levels

new level = old level pairs (as a named character vector)



```
gss_cat %>%
   filter(!is.na(tvhours)) %>%
    mutate(partyid = fct_recode(partyid,
    "Republican, strong"    = "Strong republican",
    "Republican, weak"      = "Not str republican",
    "Independent, near rep" = "Ind,near rep",
    "Independent, near dem" = "Ind,near dem",
    "Democrat, weak"        = "Not str democrat",
    "Democrat, strong"      = "Strong democrat")) %>%
   group_by(partyid) %>%
   summarise(tvhours = mean(tvhours)) %>%
   ggplot(aes(tvhours, fct_reorder(partyid, tvhours))) +
     geom_point() + labs(y = "partyid")
```




Democrat, strong
No answer
Independent
Democrat, weak
Independent, near dem
Other party
Independent, near rep
Republican, strong
Republican, weak
Don't know
partyid
liberal
liberal
liberal
conservative
conservative
conservative
2.0
2.5
3.0
3.5
tvhours
2. How can we combine these groups?

Collapsing levels

fct_collapse()

Collapses multiple levels into single levels

```
fct_collapse(f, Liberal = c("Democrat, strong",
                            "Democrat, weak"))
```

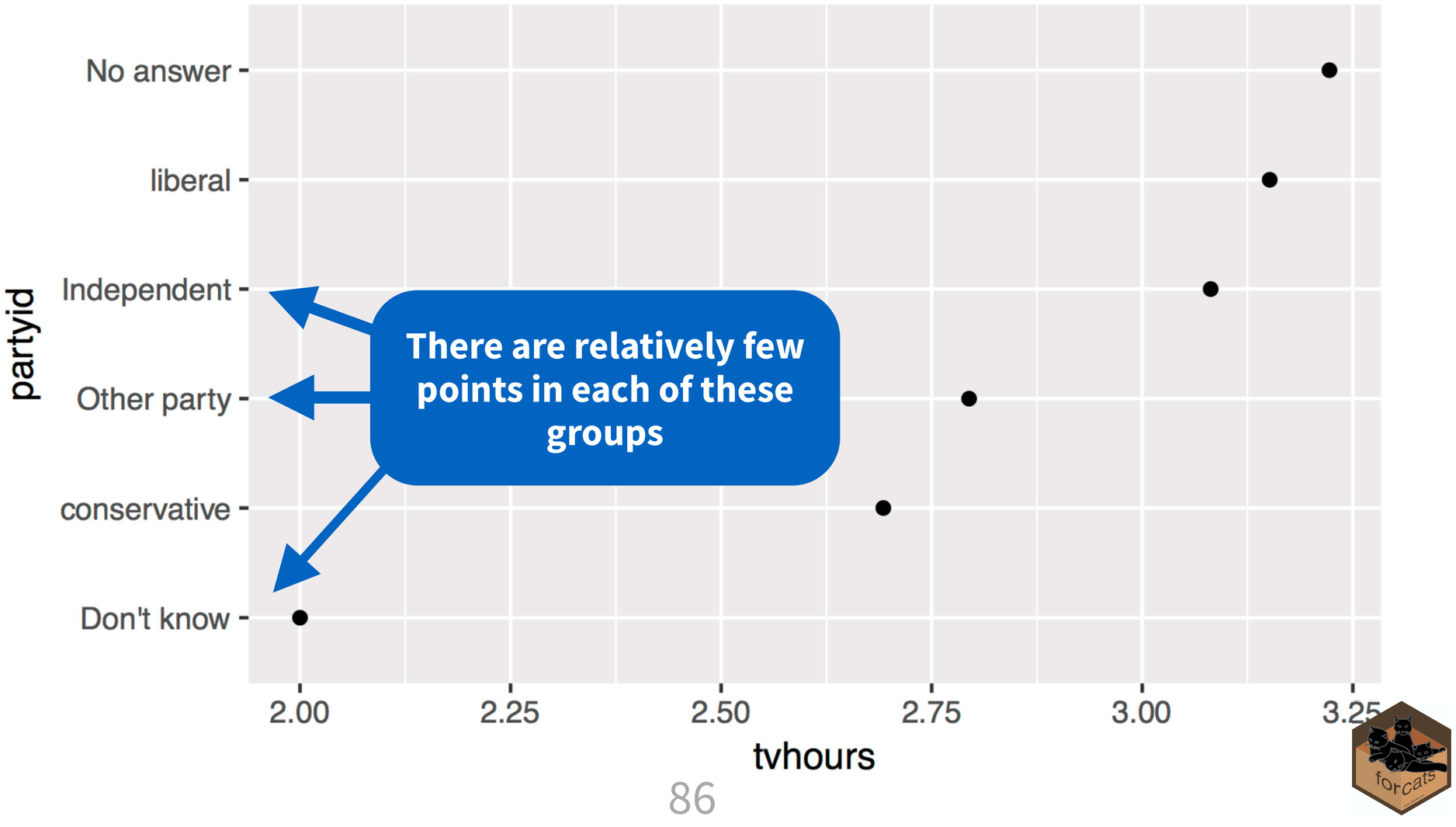
factor with levels

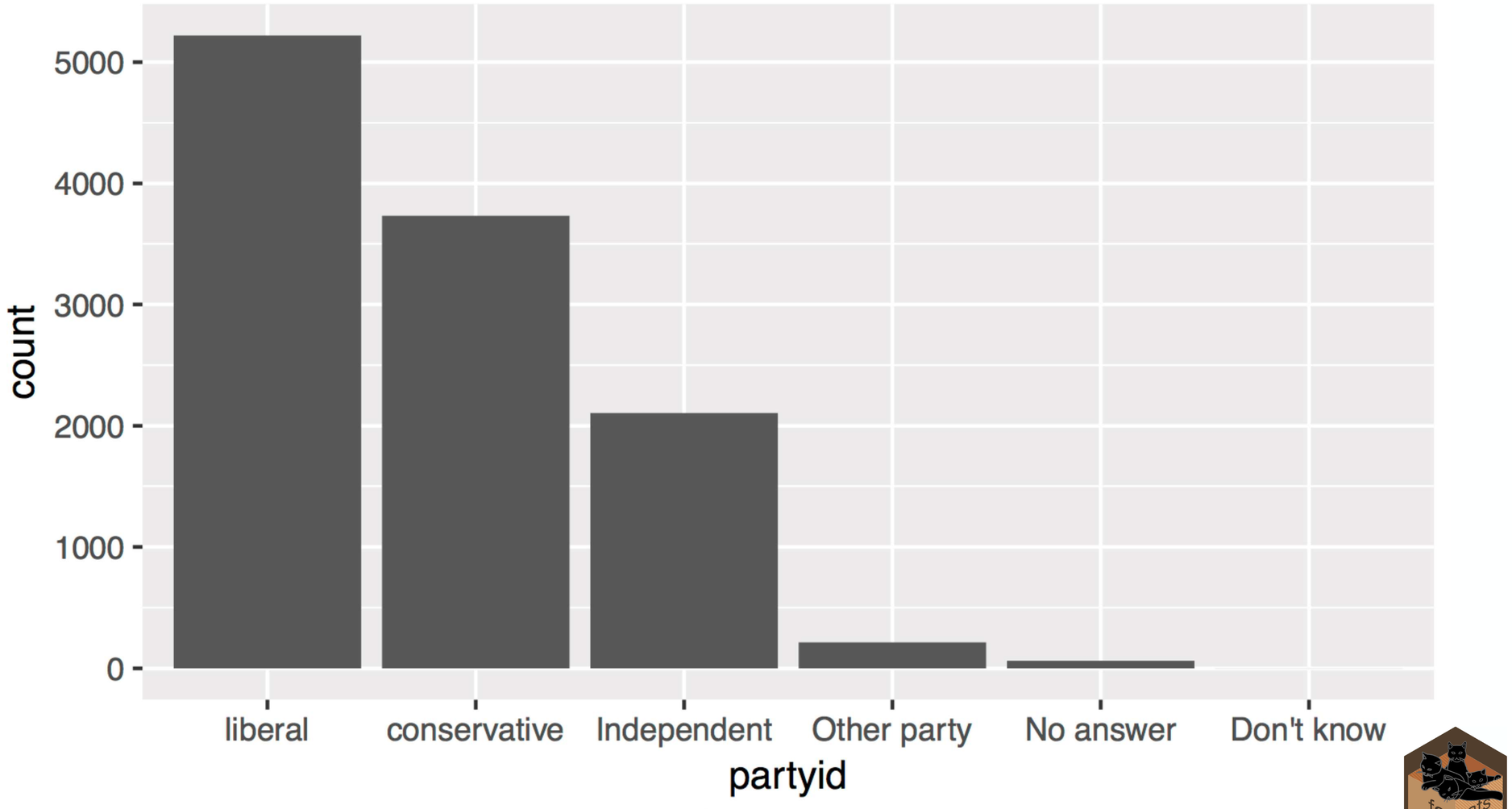
named arguments set to a character vector (levels in the vector will be collapsed to the name of the argument)



```
gss_cat %>%
   filter(!is.na(tvhours)) %>%
    mutate(partyid = fct_collapse(partyid,
      conservative = c("Strong republican",
                       "Not str republican",
                       "Ind,near rep"),
      liberal = c("Strong democrat",
                  "Not str democrat",
                  "Ind,near dem"))) %>%
  group_by(partyid) %>%
  summarise(tvhours = mean(tvhours)) %>%
  ggplot(aes(tvhours, fct_reorder(partyid, tvhours))) +
    geom_point() + labs(y = "partyid")
```




No answer
liberal
Independent
Other party
conservative
Don't know
partyid
2.00
2.25
2.50
2.75
3.00
3.25
tvhours
There are relatively few points in each of these groups


5000
4000
3000
2000
1000
0
count
liberal
conservative
Independent
Other party
No answer
Don't know
partyid

fct_lump()

Collapses levels with fewest values into a single level. Collapses as many levels as possible such that the new level is still the smallest.

```
fct_lump(f, other_level = "Other", …)
```

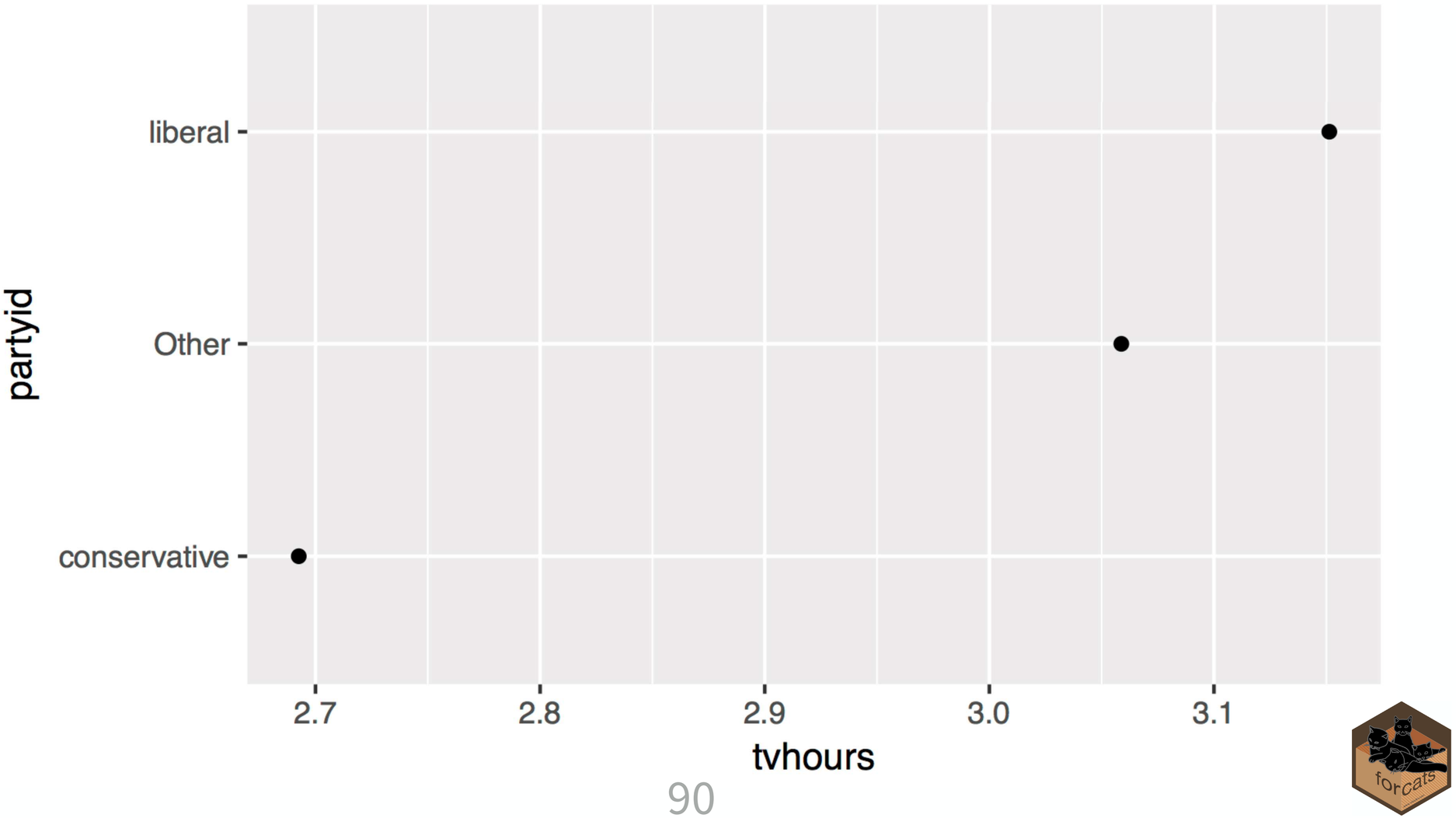
factor with levels

name of new level



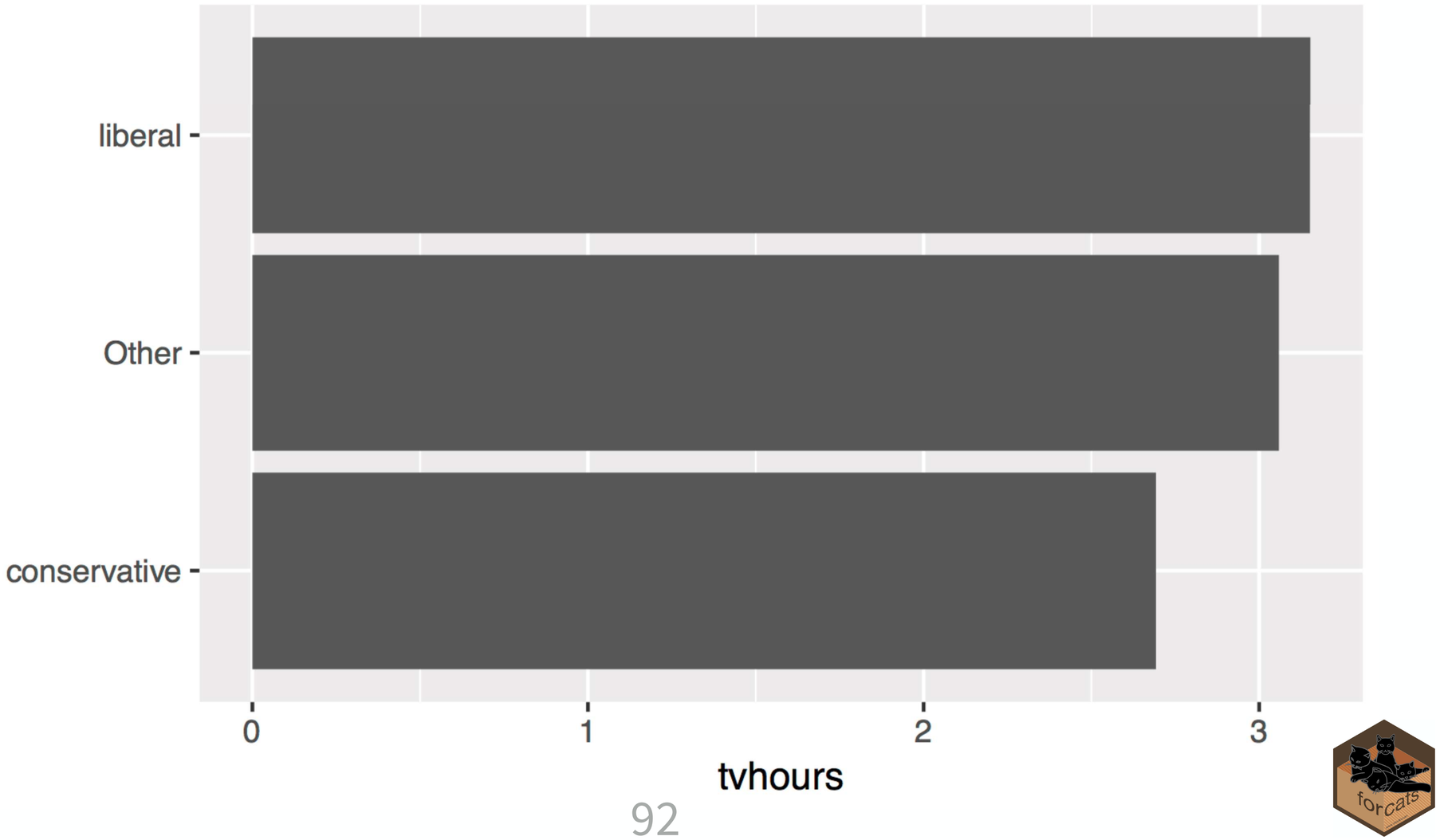
```
gss_cat %>%
  filter(!is.na(tvhours)) %>%
  mutate(partyid = partyid %>%
    fct_collapse(
      conservative = c("Strong republican",
                       "Not str republican", "Ind,near rep"),
      liberal = c("Strong democrat", "Not str democrat",
                  "Ind,near dem")) %>%
    fct_lump()
) %>%
group_by(partyid) %>%
summarise(tvhours = mean(tvhours)) %>%
ggplot(aes(tvhours, fct_reorder(partyid, tvhours))) +
  geom_point() + labs(y = "partyid")
```




partyid
liberal
Other
conservative
2.7
2.8
2.9
3.0
3.1
tvhours

```
gss_cat %>%
  filter(!is.na(tvhours)) %>%
  mutate(partyid = partyid %>%
    fct_collapse(
      conservative = c("Strong republican",
                       "Not str republican", "Ind,near rep"),
      liberal = c("Strong democrat", "Not str democrat",
                  "Ind,near dem")) %>%
    fct_lump()
) %>%
group_by(partyid) %>%
summarise(tvhours = mean(tvhours)) %>%
ggplot(aes(fct_reorder(partyid, tvhours), tvhours)) +
  geom_col() + labs(x = "partyid") + coord_flip()
```




partyid
liberal
Other
conservative
0
1
2
3
tvhours

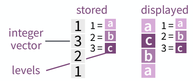
factors

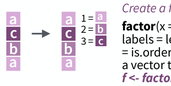
Factors with forcats : : **CHEAT SHEET**

The **forcats** package provides tools for working with factors, which are R's data structure for categorical data.

Factors

R represents categorical data with factors. A **factor** is an integer vector with a **levels** attribute that stores a set of mappings between integers and categorical values. When you view a factor, R displays not the integers, but the values associated with them.

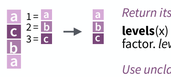




Create a factor with factor()

factor(x = character(), levels, labels = levels, exclude = NA, ordered = is.ordered(x), nmax = NA) Convert a vector to a factor. Also **as_factor**.

```
f <- factor(c("a", "c", "b", "a"),
            levels = c("a", "b", "c"))
```

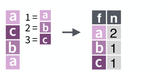


Return its levels with levels()

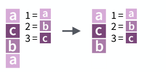
levels(x) Return/set the levels of a factor. *levels(f); levels(f) <- c("x","y","z")*

Use unclass() to see its structure

Inspect Factors

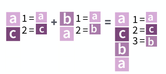


fct_count(f, sort = FALSE) Count the number of values with each level. *fct_count(f)*



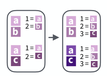
fct_unique(f) Return the unique values, removing duplicates. *fct_unique(f)*

Combine Factors



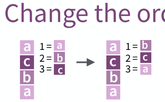
fct_c(…) Combine factors with different levels.

```
f1 <- factor(c("a", "c"))
f2 <- factor(c("b", "a"))
fct_c(f1, f2)
```

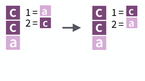


fct_unify(fs, levels = lvls_union(fs)) Standardize levels across a list of factors. *fct_unify(list(f2, f1))*

Change the order of levels



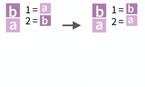
fct_relevel(.f, ..., after = 0L) Manually reorder factor levels.
fct_relevel(f, c("b", "c", "a"))



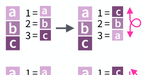
fct_infreq(f, ordered = NA) Reorder levels by the frequency in which they appear in the data (highest frequency first).

```
f3 <- factor(c("c", "c", "a"))
```

fct_infreq(f3)



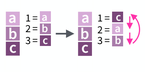
fct_inorder(f, ordered = NA) Reorder levels by order in which they appear in the data.
fct_inorder(f2)



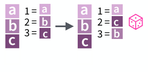
fct_rev(f) Reverse level order.

```
f4 <- factor(c("a","b","c"))
```

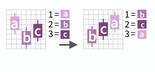
fct_rev(f4)



fct_shift(f) Shift levels to left or right, wrapping around end.
fct_shift(f4)



fct_shuffle(f, n = 1L) Randomly permute order of factor levels.
fct_shuffle(f4)

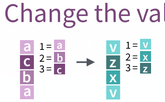


fct_reorder(.f, .x, .fun=median, ..., .desc = FALSE) Reorder levels by their relationship with another variable.
boxplot(data = iris, Sepal.Width ~ fct_reorder(Species, Sepal.Width))



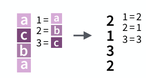
fct_reorder2(.f, .x, .y, .fun = last2, ..., .desc = TRUE) Reorder levels by their final values when plotted with two other variables.
ggplot(data = iris, aes(Sepal.Width, Sepal.Length, color = fct_reorder2(Species, Sepal.Width, Sepal.Length))) + geom_smooth()

Change the value of levels

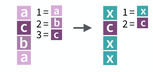


fct_recode(.f, ...) Manually change levels. Also **fct_relabel** which obeys purrr::map syntax to apply a function or expression to each level.

```
fct_recode(f, v = "a", x = "b", z = "c")
fct_relabel(f, ~ paste0("x", .x))
```



fct_anon(f, prefix = "")) Anonymize levels with random integers. *fct_anon(f)*



fct_collapse(.f, ...) Collapse levels into manually defined groups.
fct_collapse(f, x = c("a", "b"))

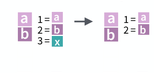


fct_lump(f, n, prop, w = NULL, other_level = "Other", ties.method = c("min", "average", "first", "last", "random", "max")) Lump together least/most common levels into a single level. Also **fct_lump_min**.
fct_lump(f, n = 1)



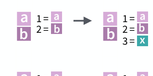
fct_other(f, keep, drop, other_level = "Other") Replace levels with "other."
fct_other(f, keep = c("a", "b"))

Add or drop levels

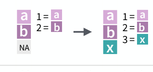


fct_drop(f, only) Drop unused levels.

```
f5 <- factor(c("a","b"),c("a","b","x"))
f6 <- fct_drop(f5)
```



fct_expand(f, …) Add levels to a factor. *fct_expand(f6, "x")*



fct_explicit_na(f, na_level="(Missing)") Assigns a level to NAs to ensure they appear in plots, etc.
fct_explicit_na(factor(c("a", "b", NA)))

RStudio® is a trademark of RStudio, Inc. • CC BY SA RStudio • info@rstudio.com • 844-448-1212 • rstudio.com • Learn more at **forcats.tidyverse.org** • Diagrams inspired by **@LVaudor** • forcats 0.3.0 • Updated: 2019-02